

**Informe de Práctica Guiada de OpenCV: Representación, Transformación y Análisis de
Imágenes Digitales**

Julian Esteban Ballesteros Ortiz

Juan Diego Walteros Cortes

Universidad de Cundinamarca,

Especialización Inteligencia Artificial

CADI Percepción Computacional - CAD2202023102

MÓNICA MARÍA FONSECA VIGOYA

lunes, 5 de octubre de 2026

Resumen

Este informe documenta la práctica guiada de la Semana 7 del CADI Percepción Computacional, cuyo propósito fue comprender cómo una imagen digital se representa como datos numéricos y aplicar operaciones básicas de procesamiento con OpenCV en Python. El mismo flujo de trabajo se ejecutó sobre dos fotografías propias de contenido y resolución muy distintos: cinco figuras pequeñas sobre un parlante (3000×4000 píxeles) y un escritorio de trabajo (8000×6000 píxeles). Se aplicaron carga e inspección, conversión de color, redimensionamiento, recorte, filtrado promedio, gaussiano y de mediana, umbralización global, detección de bordes de Canny, extracción de contornos con cajas delimitadoras y guardado de evidencias. Los resultados muestran que cada imagen es una matriz de valores de 8 bits por canal; que el modo BGR de OpenCV altera los colores si se visualiza sin convertir; que reducir el tamaño sin conservar la proporción deforma los objetos; que un umbral fijo depende de la iluminación; y que Canny y los contornos resaltan estructura, pero producen resultados fragmentados cuando la imagen tiene mucha textura. Se concluye que el procesamiento previo, como escalado, suavizado y elección adecuada de parámetros, determina la calidad de la información que un algoritmo puede extraer.

Palabras clave: OpenCV, procesamiento de imágenes, escala de grises, filtrado, detección de bordes, contornos.

Informe de Práctica Guiada de OpenCV: Representación, Transformación y Análisis de Imágenes Digitales

Objetivo

El objetivo de la práctica fue comprender cómo una imagen se representa computacionalmente como una matriz de números y aplicar, con la biblioteca OpenCV (Bradski, 2000), un flujo básico de procesamiento que incluye la carga y visualización de la imagen, la conversión de color, el redimensionamiento y el recorte, el filtrado para reducir ruido, la umbralización, la detección de bordes, la localización de contornos y el guardado de los resultados como evidencia.

Para poder comparar cómo influyen el contenido, la resolución y la iluminación, el mismo código se ejecutó en dos cuadernos de Google Colab (`test.ipynb` y `test_2.ipynb`) con fotografías propias. La Imagen 1 muestra cinco figuras pequeñas de colores sobre un parlante negro frente a una cortina clara; la Imagen 2 muestra un escritorio oscuro con un monitor, un teclado rojo y un tapete frente a una pared blanca con relieve. Además del registro de resultados, el informe busca explicar qué hizo cada operación y qué información permitió observar.

Desarrollo

La práctica se desarrolló en Google Colab con Python, OpenCV 4.14.0, NumPy 2.1.3 y Matplotlib. Como OpenCV carga las imágenes en orden de canales azul, verde y rojo (BGR) y Matplotlib espera rojo, verde y azul (RGB), se definió una función de apoyo, `mostrar`, que convierte el espacio de color antes de graficar. A continuación se presentan, en orden, los ejercicios realizados con fragmentos de código breves; las Figuras 1 a 7 comparan siempre la

Imagen 1 (a) con la Imagen 2 (b), y la interpretación detallada de cada transformación se desarrolla en la sección de Análisis.

Carga e inspección de la imagen

Cada fotografía se cargó con `cv2.imread` y se inspeccionó como un arreglo de datos: forma, tipo, valores mínimo y máximo, y el valor del píxel ubicado en el centro.

```
img = cv2.imread(ruta_imagen)
print(img.shape, img.dtype, img.min(), img.max())
alto, ancho = img.shape[:2]
print(img[alto//2, ancho//2]) # píxel central (B, G, R)
```

Figura 1

Imágenes originales cargadas con OpenCV y mostradas en RGB

Imagen cargada: /content/Imagen.jpeg



Imagen cargada: /content/20260411_131147.jpg.jpeg



Nota. (a) Imagen 1, en orientación vertical. (b) Imagen 2, en orientación horizontal. Elaboración propia con OpenCV y Matplotlib.

Tabla 1

Propiedades de las imágenes cargadas

Propiedad	Imagen 1	Imagen 2
Forma (alto, ancho, canales)	(4000, 3000, 3)	(6000, 8000, 3)

Tipo de dato	uint8	uint8
Valor mínimo y máximo	0 y 255	0 y 255
Total de píxeles	12 000 000	48 000 000
Píxel central (B, G, R)	[39, 16, 24]	[33, 28, 30]

Nota. Valores obtenidos con `img.shape`, `img.dtype`, `img.min()`, `img.max()` e `img[alto//2, ancho//2]`.

Conversión de color

Se convirtió la imagen a RGB para su visualización correcta y a escala de grises con `cv2.cvtColor`. Como contraste, también se mostró la imagen BGR sin convertir.

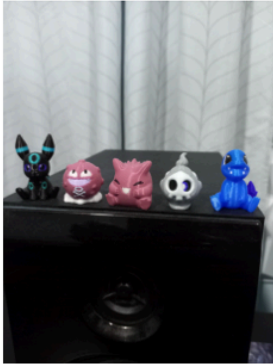
```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gris    = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Figura 2

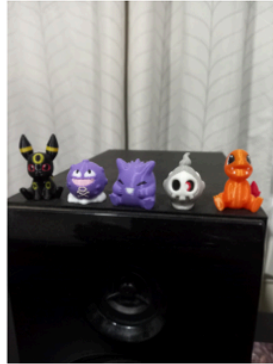
Imagen en BGR visto directo, convertida a RGB y en escala de grises

(a) Imagen 1: figuras sobre el parlante

OpenCV BGR visto directo



Convertida a RGB



Escala de grises

**(b) Imagen 2: escritorio de trabajo**

OpenCV BGR visto directo



Convertida a RGB



Escala de grises



Nota. (a) Imagen 1. (b) Imagen 2. Elaboración propia con OpenCV y Matplotlib.

Redimensionamiento y recorte

La imagen se redujo a 320×210 píxeles con `cv2.resize` y se extrajo una región central equivalente al 50 % del alto y del ancho mediante segmentación del arreglo (*slicing*) de NumPy.

```
redimensionada = cv2.resize(img, (320, 210))
y1, y2 = int(h*0.25), int(h*0.75)
x1, x2 = int(w*0.25), int(w*0.75)
recorte = img[y1:y2, x1:x2]
```

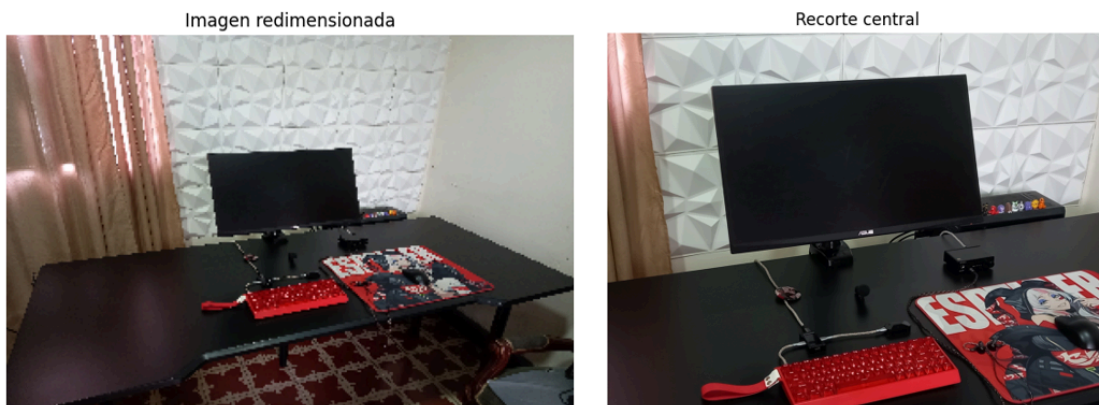
Figura 3

Imagen redimensionada a 320×210 píxeles y recorte de la zona central

(a) Imagen 1: redimensionada (izq.) y recorte central (der.)



(b) Imagen 2: redimensionada (izq.) y recorte central (der.)



Nota. Elaboración propia con OpenCV y Matplotlib.

Tabla 2

Dimensiones (alto, ancho, canales) antes y después de redimensionar y recortar

Imagen	Original	Redimensionada	Recorte
Imagen 1	(4000, 3000, 3)	(210, 320, 3)	(2000, 1500, 3)
Imagen 2	(6000, 8000, 3)	(210, 320, 3)	(3000, 4000, 3)

Nota. Dimensiones obtenidas con el atributo `shape` de cada arreglo.

Filtrado y reducción de ruido

Se compararon tres filtros de suavizado con una ventana de 7×7 píxeles: el filtro promedio, el filtro gaussiano y el filtro de mediana.

```
blur_promedio    = cv2.blur(img, (7, 7))  
blur_gaussiano  = cv2.GaussianBlur(img, (7, 7), 0)  
blur_mediana    = cv2.medianBlur(img, 7)
```

Figura 4

Filtros de suavizado aplicados a la Imagen 1



Nota. De izquierda a derecha: original, filtro promedio, filtro gaussiano y filtro de mediana.

Elaboración propia con OpenCV y Matplotlib.

Figura 5

Filtros de suavizado aplicados a la Imagen 2

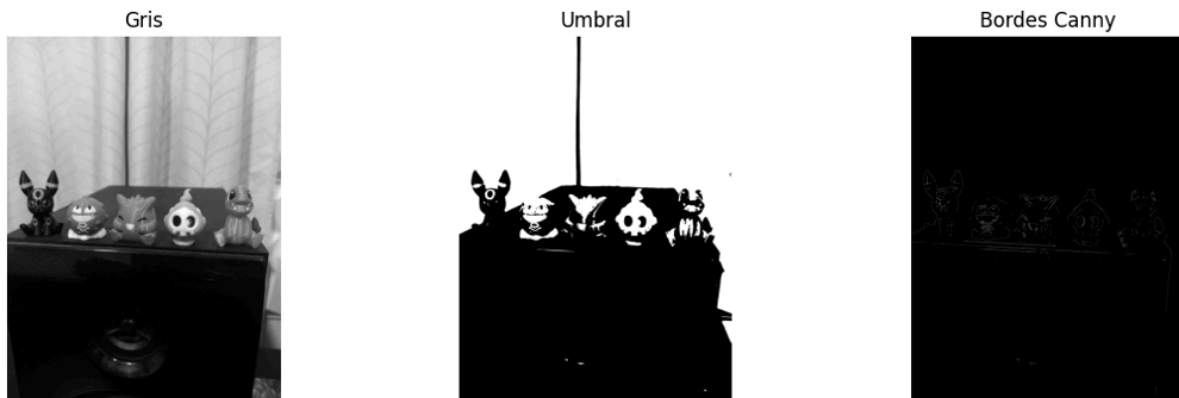
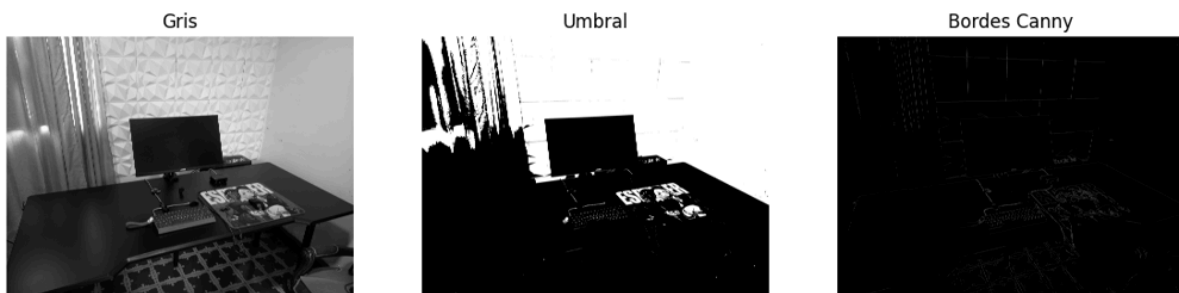


Nota. Disposición: original, promedio (arriba); gaussiano, mediana (abajo). Elaboración propia con OpenCV y Matplotlib.

Umbralización y detección de bordes

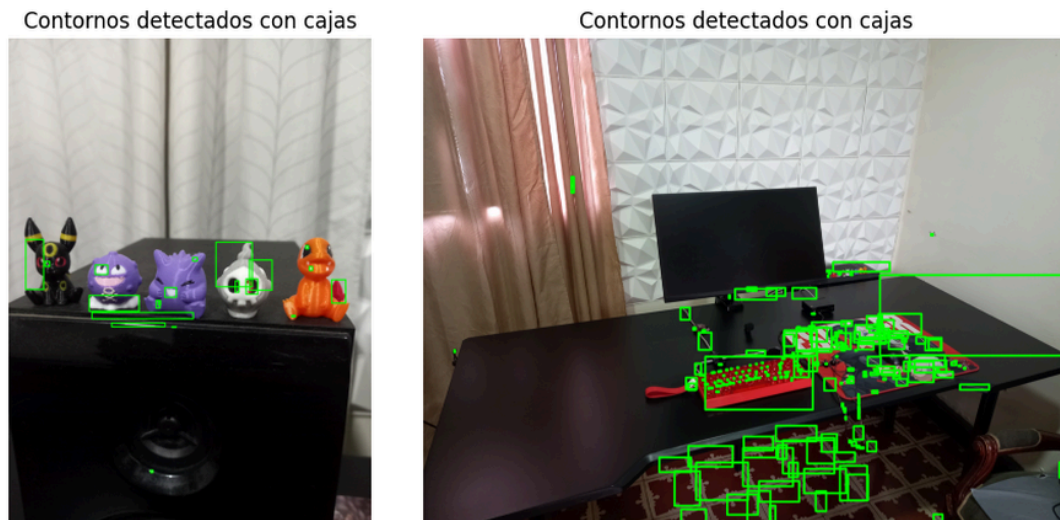
Sobre la imagen en gris suavizada con un filtro gaussiano de 5×5 se aplicó un umbral global de 127 y el detector de bordes de Canny (Canny, 1986) con umbrales de histéresis de 80 y 160.

```
gris_suave = cv2.GaussianBlur(gris, (5, 5), 0)
_, umbral = cv2.threshold(gris_suave, 127, 255, cv2.THRESH_BINARY)
bordes = cv2.Canny(gris_suave, 80, 160)
```

Figura 6*Imagen en gris, umbralización binaria y bordes de Canny***(a) Imagen 1: figuras sobre el parlante****(b) Imagen 2: escritorio de trabajo***Nota.* (a) Imagen 1. (b) Imagen 2. Elaboración propia con OpenCV y Matplotlib.**Contornos y cajas delimitadoras**

A partir de la imagen de bordes se localizaron los contornos externos con `cv2.findContours`, un algoritmo de seguimiento de fronteras (Suzuki & Abe, 1985). Se descartaron los contornos con área menor o igual a 100 píxeles cuadrados y se dibujó una caja verde alrededor de cada contorno restante.

```
contornos, _ = cv2.findContours(bordes, cv2.RETR_EXTERNAL,
                                cv2.CHAIN_APPROX_SIMPLE)
for c in contornos:
    if cv2.contourArea(c) > 100:
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(resultado, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

Figura 7*Contornos detectados con cajas delimitadoras*

Nota. (a) Imagen 1. (b) Imagen 2. Las cajas verdes marcan los contornos con área mayor a 100 píxeles cuadrados; se concentran en las figuras y en las zonas de mayor textura, como el teclado, el tapete y la alfombra, y ninguna encierra un objeto completo. Elaboración propia con OpenCV y Matplotlib.

Tabla 3*Resultados de la detección de contornos*

Métrica	Imagen 1	Imagen 2
Contornos detectados	542	1704
Contornos con área > 100 px ²	20	210
Proporción que supera el filtro	3,7 %	12,3 %
Áreas de los primeros diez filtrados (px ²)	100,5 a 1079,5	102,0 a 288,5

Nota. Las áreas corresponden a los diez primeros contornos filtrados que imprime el cuaderno.

Guardado de evidencias

Por último, las ocho imágenes intermedias (original, gris, redimensionada, recorte, desenfoque gaussiano, umbral, bordes y contornos) se guardaron en formato PNG, que no pierde información, dentro de la carpeta `resultados_opencv` con `cv2.imwrite`, y se comprimieron en un archivo `.zip` para su descarga y uso en este informe.

Análisis

La imagen como matriz de datos

Una imagen con forma `(alto, ancho, 3)` es un arreglo tridimensional de NumPy: el primer índice recorre las filas, el segundo las columnas y el tercero los tres canales de color de cada píxel. En OpenCV el orden de esos canales es azul, verde y rojo, y cada valor es un entero de 8 bits (`uint8`) entre 0 y 255 que indica la intensidad de ese color. Así, la Imagen 1 contiene 12 000 000 de píxeles y 36 000 000 de valores, y la Imagen 2 contiene 48 000 000 de píxeles y 144 000 000 de valores, es decir, cuatro veces más datos. Los píxeles centrales, `[39, 16, 24]` y `[33, 28, 30]`, tienen valores bajos en los tres canales, lo que corresponde a tonos casi negros y concuerda con lo que se ve en las fotografías: el centro cae sobre el parlante negro y sobre la mesa oscura. Que el mínimo sea 0 y el máximo 255 indica que ambas imágenes aprovechan todo el rango dinámico, con zonas muy oscuras y zonas muy claras.

Conversión de color

Al mostrar directamente la imagen BGR con Matplotlib se intercambian los canales rojo y azul (Figura 2). En la Imagen 1, la figura naranja aparece azul y las figuras moradas aparecen rosadas; en la Imagen 2, el rojo del teclado y del tapete se desplaza hacia el azul por la misma razón. Esta operación permitió observar que los números no cambian, sino su interpretación: el

orden de los canales es una convención y confundirla produce colores incorrectos sin generar ningún error. La conversión a escala de grises reduce cada píxel de tres valores a uno, una combinación ponderada de los canales que representa la luminosidad, por lo que la imagen ocupa un tercio de los datos. Se conservan formas y contrastes de brillo, pero se pierde la información cromática: en la Imagen 1, las figuras morada y naranja quedan en tonos de gris medio similares y ya no se distinguen por color. La escala de grises es el paso de partida para los umbrales y para Canny, que trabajan con intensidades y no con colores.

Redimensionamiento y recorte

El redimensionamiento redujo la Imagen 1 de 12 000 000 a 67 200 píxeles (unas 179 veces menos) y la Imagen 2 de 48 000 000 a 67 200 (unas 714 veces menos). Esto disminuye el costo de cómputo y sirve para preparar imágenes de tamaño uniforme para un modelo, pero elimina detalle fino: en la Imagen 2 se aprecian bordes escalonados en la mesa y el monitor (Figura 3b). Como el tamaño de destino, 320×210 , no respeta la proporción original, la Imagen 1 pasó de vertical a horizontal y sus figuras se ven aplastadas, mientras que la Imagen 2 se estiró levemente en horizontal. Esta deformación es una advertencia práctica: conviene calcular una de las dimensiones a partir de la otra o rellenar los bordes. El recorte, en cambio, es una selección de filas y columnas del arreglo; no interpola ni cambia la resolución, solo descarta datos. Al conservar el 25 % de los píxeles, el recorte de la Imagen 1 dejó a la vista las tres figuras centrales con todo su detalle y eliminó los extremos, y el de la Imagen 2 aisló el monitor, el teclado y parte del tapete. El recorte permite entonces centrar el análisis en una región de interés sin perder calidad.

Filtrado y reducción de ruido

Los tres filtros sustituyen cada píxel por un valor calculado a partir de su vecindario de 7×7 píxeles. El filtro promedio asigna el mismo peso a todos los vecinos y tiende a difuminar por igual bordes y ruido; el gaussiano pondera más los vecinos cercanos, por lo que suaviza con mayor naturalidad; y el de mediana toma el valor central del vecindario, lo que elimina bien el ruido de tipo impulsivo (puntos aislados muy claros u oscuros) y conserva mejor los bordes. En las Figuras 4 y 5 las diferencias entre los resultados y la imagen original son difíciles de percibir. La razón es la escala: una ventana de 7 píxeles abarca apenas el 0,23 % del ancho de la Imagen 1 (3000 píxeles) y menos del 0,1 % del ancho de la Imagen 2 (8000 píxeles), y además las figuras se muestran reducidas en el informe. Esto indica que el tamaño de la ventana debe elegirse en relación con la resolución de la imagen y con el tamaño del ruido o de la textura que se desea atenuar. Su utilidad principal en este flujo es preparar la imagen para Canny, ya que reduce variaciones diminutas que serían interpretadas como bordes.

Umbralización y detección de bordes

La umbralización binaria convierte la imagen en gris en blanco y negro: todo píxel mayor que 127 pasa a 255 y el resto a 0. Permite observar qué zonas son claras y cuáles oscuras, pero su resultado depende de la iluminación. En la Imagen 1, la cortina clara queda blanca y el parlante negro, de modo que las figuras solo se recortan en las partes más brillantes (por ejemplo, la figura blanca) y las figuras moradas se fragmentan. En la Imagen 2, la pared y las cortinas quedan blancas, la mesa y el monitor negros, y casi todo el detalle de la zona oscura desaparece (Figura 6). Un umbral único, por tanto, no distingue objetos que tengan un brillo similar al del fondo y sería mejor un umbral adaptativo.

Al aplicar Canny cambia por completo la naturaleza de la imagen: pasa de una matriz de intensidades continuas a una imagen casi negra en la que sólo los píxeles donde la intensidad cambia con fuerza quedan en blanco. Se vuelve más visible la información estructural, es decir, los contornos de las figuras, el borde del parlante, el marco del monitor, las teclas y el patrón de la alfombra, y se descartan el color y las regiones uniformes. Las líneas resultan delgadas y tenues en la Figura 6 porque tienen un píxel de grosor sobre imágenes muy grandes. También se nota la sensibilidad a la textura: la Imagen 2, con pared en relieve, cortinas y alfombra estampada, genera muchos más bordes que la Imagen 1, cuyo fondo es más homogéneo. Los umbrales 80 y 160 funcionan como histéresis: un borde se acepta si su gradiente supera 160, o si supera 80 y está conectado con uno fuerte (Canny, 1986).

Contornos y cajas delimitadoras

`findContours` transforma los píxeles blancos de la imagen de bordes en listas de coordenadas, que pueden medirse (área) y encerrarse en una caja. Esto conecta los píxeles con posibles objetos o zonas de interés, pero el resultado depende de la calidad de los bordes. Se detectaron 542 contornos en la Imagen 1 y 1704 en la Imagen 2, y solo 20 y 210, respectivamente, superaron el umbral de área de 100 píxeles cuadrados (Tabla 3). Ese umbral es muy pequeño para imágenes de 12 y 48 millones de píxeles, por lo que una parte de las cajas corresponde a fragmentos de textura y no a objetos. Las cajas de la Figura 7 aparecen sobre la base de las figuras centrales y la figura blanca en la Imagen 1, y sobre el teclado, el tapete, la base del monitor y la alfombra en la Imagen 2, pero ninguna encierra un objeto completo. La causa es que los bordes de Canny rara vez forman curvas cerradas: cada segmento genera su propio contorno. La práctica permitió comprobar que detectar bordes no equivale a detectar objetos.

Conclusiones

La práctica permitió aprender que una imagen es, ante todo, un arreglo numérico y que cada operación de OpenCV es una transformación de esos valores: la conversión de color reorganiza o reduce los canales, el redimensionamiento y el recorte cambian la cantidad de datos, los filtros promedian vecindarios, la umbralización clasifica por intensidad y Canny conserva solo los cambios abruptos. Al aplicar el mismo código a dos fotografías se comprobó que los resultados dependen del contenido, de la resolución y de la iluminación, y que los parámetros no son universales.

Las principales dificultades fueron cuatro. Primera, el orden BGR de OpenCV, que cambia los colores al visualizar sin convertir. Segunda, el tamaño de las imágenes, de 12 y 48 millones de píxeles, con el que filtros de 5×5 o 7×7 y un umbral de área de 100 píxeles resultan casi imperceptibles. Tercera, el umbral fijo de 127, que no separa bien objetos oscuros de fondos oscuros ni claros de fondos claros. Cuarta, los contornos fragmentados, que no delimitan objetos completos. Como operación más útil se puede señalar la conversión a escala de grises seguida de suavizado y Canny, porque simplifica los datos y hace visible la estructura de la escena, aunque por sí sola no identifica objetos.

Como recomendaciones, se sugiere reducir la imagen conservando su proporción antes de filtrar o detectar bordes; escoger el tamaño del filtro según la resolución; probar umbralización adaptativa o el método de Otsu en lugar de un valor fijo; cerrar los huecos de los bordes con operaciones morfológicas antes de buscar contornos; definir el área mínima como fracción del tamaño de la imagen; y dibujar las cajas con mayor grosor para que sean visibles en el informe. Con estos ajustes, el flujo practicado es una base adecuada para tareas posteriores de reconocimiento de objetos.

Referencias

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120–125.

Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6), 679–698.

<https://doi.org/10.1109/TPAMI.1986.4767851>

Suzuki, S., & Abe, K. (1985). Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1), 32–46.

[https://doi.org/10.1016/0734-189X\(85\)90016-7](https://doi.org/10.1016/0734-189X(85)90016-7)